LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 5 – RELASI ANTAR KELAS



Ardian Nafis Samudra

103132400012

PS1SD-05-reg-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

- **Diagram Kelas (Class Diagram):** Representasi visual dalam Pemrograman Berorientasi Objek untuk memodelkan struktur kelas, atribut, *access modifier* (+ untuk *public*, - untuk *private*), serta *method* di dalam sistem.
- **Asosiasi (Association):** Hubungan interaksi antara dua kelas independen yang saling berkomunikasi dan menggunakan *method* satu sama lain tanpa saling memiliki sebagai atribut permanen.
- **Agregasi (Aggregation):** Hubungan antar kelas yang merepresentasikan kondisi bagian (*part*) dan keseluruhan (*whole*), di mana objek bagian (*part*) dapat diciptakan dan berdiri sendiri secara independen.
- **Komposisi (Composition):** Bentuk khusus agregasi yang memiliki ikatan lebih kuat, di mana objek bagian (*part*) merupakan bagian internal yang siklus hidupnya terikat penuh dan bergantung pada objek utamanya.
- **Package dan Pengorganisasian Kelas:** Pengelompokan kelas ke dalam folder atau *package* terpisah (GuidedAsosiasi, GuidedAgregasi, GuidedKomposisi, dan Unguided) guna menjaga keteraturan struktur proyek Java.

B. Guided

1. GuidedAgregasi (Mahasiswa.java, Jurusan.java, & main.java)

```
package com.mycompany.praktikummgg05.GuidedAgregasi;

public class Mahasiswa {
    private String NIM, Nama;

    public Mahasiswa (String NIM, String Nama) {
        this.NIM = NIM;
        this.Nama = Nama;
    }
    //getter
    public String GetNIM() {
        return NIM;
    }

    public String GetNama () {
        return Nama;
    }

    //setter
    public void setNIM(String NIM){
        this.NIM = NIM;
```

```
    }

    public void setNama(String Nama){
        this.Nama = Nama;
    }

    String getNIM() {
        throw new UnsupportedOperationException("Not supported yet."); // Generated from
nbfs://nbhost/SystemFileSystem/Templates/Classes/Code/GeneratedMethodBody
    }

    String getNama() {
        throw new UnsupportedOperationException("Not supported yet."); // Generated from
nbfs://nbhost/SystemFileSystem/Templates/Classes/Code/GeneratedMethodBody
    }
            }
```

```
package com.mycompany.praktikummgg05.GuidedAgregasi;

public class Jurusan {
    private String KodeJurusan, NamaJurusan;
    private Mahasiswa[] Daftar = new Mahasiswa[10];
    private static int JmlMhs = 0;

    public Jurusan (String kode, String nama) {
        this.KodeJurusan = kode;
        this.NamaJurusan = nama;
    }

    // Ubah menjadi AddMahasiswa (dengan huruf A kapital)
    public void AddMahasiswa (Mahasiswa m) {
        this.Daftar[JmlMhs] = m;
        this.JmlMhs++;
    }

    // Ubah menjadi DisplayMahasiswa (dengan huruf D kapital)
    public void DisplayMahasiswa () {
        int i;
        System.out.println("Kode Jurusan : " + this.KodeJurusan);
        System.out.println("Nama Jurusan : " + this.NamaJurusan);
        System.out.println("Daftar Mahasiswa :");
        for (i = 0; i < JmlMhs; i++)
            System.out.println(Daftar[i].GetNIM() + " " + Daftar[i].GetNama());
    }
}
```

```
package com.mycompany.praktikummgg05.GuidedAgregasi;

public class main {
    public static void main(String[] args) {
        runGuided2();
    }

    public static void runGuided2() {
        Mahasiswa m1 = new Mahasiswa("189113400148", "Gatot");
        Mahasiswa m2 = new Mahasiswa("189113400149", "Jarwo");

        Jurusan prodi = new Jurusan("SD", "Sains Data");

        prodi.AddMahasiswa(m1);
        prodi.AddMahasiswa(m2);

        prodi.DisplayMahasiswa();
    }
}
```

Output :

```
Kode Jurusan : SD
Nama Jurusan : Sains Data
Daftar Mahasiswa :
189113400148 Gatot
189113400149 Jarwo
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  6.134 s
Finished at: 2026-10-17T15:16:06+07:00
------------------------------------------------------------------------
```

2. GuidedAsosiasi (Mahasiswa.java & Dosen.java)

```
package com.mycompany.praktikummgg05.GuidedAsosiasi;

public class Mahasiswa {
    private String nim;
    private String nama;
    private int nilai; // Tambahkan atribut nilai

    public void setNama (String nama) {
        this.nama = nama;
```

```
    }

    public void setNim (String nim) {
        this.nim = nim;
    }

    public void setNilai (int nilai) {
        this.nilai = nilai;
    }

    public String getNama () {
        return this.nama;
    }

    public String getNim(){
        return this.nim;
    }

    public int getNilai() {
        return this.nilai;
    }
            }
```

```
package com.mycompany.praktikummgg05.GuidedAsosiasi;

public class Dosen {
    private String kodeDosen;
    private String namaDosen;

    //Setter
    public void setKodeDosen (String kodeDosen) {
        this.kodeDosen = kodeDosen;
    }

    public void setNamaDosen (String namaDosen) {
        this.namaDosen = namaDosen;
    }

    //Getter
    public String getKodeDosen () {
        return this.kodeDosen;
    }

    public String getNamaDosen () {
        return this.namaDosen;
    }
```

```
    public void giveScore (Mahasiswa s, int nilai) {
        s.setNilai (nilai);
    }

    public int getScore (Mahasiswa s) {
        return s.getNilai();
    }

    public static void main(String[] args) {
        Mahasiswa m = new Mahasiswa();
        m.setNim("130118383");
        m.setNama ("Budi");

        Dosen d = new Dosen();
        d.giveScore (m, 90);

        System.out.println("Nim: "+m.getNim());
        System.out.println("Nama: "+m.getNama());
        System.out.println("Nilai : "+d.getScore(m));
    }
}
```

Output :

```
Nim: 130118383
Nama: Budi
Nilai : 90
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  3.149 s
Finished at: 2026-10-17T15:20:25+07:00
```

3. GuidedKomposisi (Komputer.java, CPU.java, Monitor.java, Mouse.java & main.java)

```
package com.mycompany.praktikummgg05.GuidedKomposisi;

public class Komputer {
    private CPU cpu;
    private Monitor monitor;
    private Mouse mouse;

    public Komputer(String brandCpu, int kecepatanCpu, String brandMonitor, String
brandMouse, String tipeMouse){
        this.cpu = new CPU(brandCpu, kecepatanCpu);
        this.monitor = new Monitor(brandMonitor);
        this.mouse = new Mouse(brandMouse, tipeMouse);
    }

    public void displaySpecKomputer() {
        System.out.println("=== SPESIFIKASI KOMPUTER ===");
        this.cpu.displaySpec();
        this.monitor.DisplaySpec();
        this.mouse.displaySpec();
    }
            }
```

```
package com.mycompany.praktikummgg05.GuidedKomposisi;


public class CPU {
    private String merk;
    private int kecepatan;

    public CPU(String merk, int kecepatan) {
        this.merk = merk;
        this.kecepatan = kecepatan;
    }

    public void displaySpec() {
        System.out.println(this.merk + ", " + this.kecepatan + " MHz");
    }
}
```

```
package com.mycompany.praktikummgg05.GuidedKomposisi;

public class Monitor {
    private String brand;
```

```
    public Monitor(String brand){
        this.brand = brand;
    }

    public void DisplaySpec() {
        System.out.println(this.brand);
    }
}
```

```
package com.mycompany.praktikummgg05.GuidedKomposisi;

public class Mouse {
    private String brand, tipe;

    public Mouse(String brand, String tipe){
        this.brand = brand;
        this.tipe = tipe;
    }

    public void displaySpec(){
        System.out.println(this.brand + ", " + this.tipe);
    }
}
```

```
package com.mycompany.praktikummgg05.GuidedKomposisi;

public class main {
    public static void main(String[] args) {
        runGuided3();
    }

    public static void runGuided3(){
        Komputer pc = new Komputer("Intel Core i5", 3200, "LG 24 Inch", "Logitech", "Wireless");
        pc.displaySpecKomputer();
    }
}
```

Output :

```
=== SPESIFIKASI KOMPUTER ===
Intel Core i5, 3200 MHz
LG 24 Inch
Logitech, Wireless
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  2.507 s
Finished at: 2026-10-17T15:37:22+07:00
------------------------------------------------------------------------
```

C. Unguided

1. Soal Unguided

Soal — Sistem Manajemen Pesanan Cafe

Sebuah cafe membutuhkan sistem sederhana untuk mencatat dan memproses pesanan pelanggan. Dalam sistem ini, relasi antar kelas dirancang untuk mencerminkan kondisi nyata operasional cafe.

Pengerjaan soal ini dibagi menjadi 3 bagian tugas utama yang masing-masing melatih satu jenis relasi antar kelas, diikuti dengan pengujian pada kelas utama (Main).

Tugas 1: Implementasi Komposisi (Composition)

Komposisi adalah relasi di mana sebuah objek memiliki objek lain sebagai bagian internalnya yang **tidak dapat berdiri sendiri**. Siklus hidup objek internal terikat penuh pada objek utamanya.

Pada sistem cafe:

- Setiap item pesanan (ItemPesanan) pasti memiliki rincian harga (RincianHarga) yang mencatat harga dasar dan pajak (10%).
- Objek RincianHarga harus di-instansiasi langsung di dalam constructor ItemPesanan. Objek RincianHarga tidak pernah dibuat di main. Jika ItemPesanan dihapus, maka RincianHarga miliknya ikut hilang.

Tugas 2: Implementasi Agregasi (Aggregation)

Agregasi adalah relasi di mana sebuah objek menampung kumpulan objek lain yang **dapat berdiri sendiri**. Objek yang ditampung dibuat secara independen di luar, lalu dimasukkan ke dalam objek wadah.

Pada sistem cafe:

- Sebuah Pesanan menampung beberapa objek ItemPesanan menggunakan array standar.
- Objek ItemPesanan dibuat terpisah terlebih dahulu (di main), lalu dimasukkan satu per satu ke dalam Pesanan melalui method tambahItem(). Jika Pesanan dibatalkan atau dihapus, objek ItemPesanan tersebut secara konsep tetap ada.

Tugas 3: Implementasi Asosiasi (Association)

Asosiasi adalah relasi interaksi antara dua objek independen yang **saling menggunakan tetapi tidak saling memiliki**. Objek tidak menyimpan objek lain sebagai field permanen, melainkan berinteraksi melalui parameter method.

Pada sistem cafe:

- Seorang Kasir memproses pembayaran dari sebuah Pesanan.
- Class Kasir tidak menyimpan objek Pesanan sebagai field, melainkan menerima objek Pesanan sebagai parameter pada method prosesPembayaran().

Tugas 4: Eksekusi dan Pengujian di Main

Buatlah class Main untuk merangkai seluruh kelas dan menguji ketiga jenis relasi.

Alur program di main

1. Buat 1 objek Kasir (misal: "Budi").
2. Buat 3 objek ItemPesanan secara independen:
    - "Iced Americano", harga dasar: 18000
    - "Beef Croissant", harga dasar: 25000
    - "Matcha Latte", harga dasar: 22000
3. Buat 1 objek Pesanan untuk **Meja 4** (kapasitas array misal: 5 item).
4. Masukkan ketiga objek ItemPesanan ke dalam objek Pesanan menggunakan method tambahItem().
5. Tampilkan rincian pesanan dengan memanggil tampilkanPesanan() pada objek Pesanan.
6. Jalankan pemrosesan pembayaran dengan memanggil method prosesPembayaran(pesanan) dari objek Kasir.

Jawaban :

1. Unguided (RincianHarga.java, ItemPesanan.java, Pesanan.java, Kasir.java & Main.java)

```
package com.mycompany.praktikummgg05.Unguided;

public class RincianHarga {
    private double hargaDasar;
    private double pajak;

    public RincianHarga(double hargaDasar) {
        this.hargaDasar = hargaDasar;
        this.pajak = hargaDasar * 0.10;
    }

    public double getHargaDasar() {
        return this.hargaDasar;
    }

    public double getPajak() {
        return this.pajak;
    }

    public double getHargaTotal() {
        return this.hargaDasar + this.pajak;
    }
}
```

```
package com.mycompany.praktikummgg05.Unguided;

public class ItemPesanan {
    private String namaItem;
    private RincianHarga rincian;

    public ItemPesanan(String namaItem, double hargaDasar) {
        this.namaItem = namaItem;
        this.rincian = new RincianHarga(hargaDasar);
    }

    public String getNamaItem() {
        return this.namaItem;
    }

    public RincianHarga getRincian() {
        return this.rincian;
    }
```

```
}
```

```
package com.mycompany.praktikummgg05.Unguided;

public class Pesanan {
    private int nomorMeja;
    private ItemPesanan[] daftarItem;
    private int jumlahItem;

    public Pesanan(int nomorMeja, int kapasitas) {
        this.nomorMeja = nomorMeja;
        this.daftarItem = new ItemPesanan[kapasitas];
        this.jumlahItem = 0;
    }

    public void tambahItem(ItemPesanan item) {
        if (this.jumlahItem < this.daftarItem.length) {
            this.daftarItem[this.jumlahItem] = item;
            this.jumlahItem++;
        }
    }

    public void tampilkanPesanan() {
        System.out.println("=== DAFTAR PESANAN CAFE ===");
        System.out.println("Nomor Meja: " + this.nomorMeja);
        for (int i = 0; i < this.jumlahItem; i++) {
            ItemPesanan item = this.daftarItem[i];
            RincianHarga r = item.getRincian();
            System.out.println((i + 1) + ". " + item.getNamaItem() + " - Rp " + r.getHargaTotal() +
                    " (Harga: " + r.getHargaDasar() + ", Pajak: " + r.getPajak() + ")");
        }
        System.out.println();
    }

    public int getNomorMeja() {
        return this.nomorMeja;
    }

    public ItemPesanan[] getDaftarItem() {
        return this.daftarItem;
    }

    public int getJumlahItem() {
        return this.jumlahItem;
    }
}
```

```
package com.mycompany.praktikummgg05.Unguided;
```

```
public class Kasir {
   private String namaKasir;

   public Kasir(String namaKasir) {
      this.namaKasir = namaKasir;
   }

   public String getNamaKasir() {
      return this.namaKasir;
   }

   public void prosesPembayaran(Pesanan pesanan) {
      double totalSubtotal = 0;
      double totalPajak = 0;

      ItemPesanan[] items = pesanan.getDaftarItem();
      int jumlah = pesanan.getJumlahItem();

      for (int i = 0; i < jumlah; i++) {
         RincianHarga r = items[i].getRincian();
         totalSubtotal += r.getHargaDasar();
         totalPajak += r.getPajak();
      }

      double totalBayar = totalSubtotal + totalPajak;

      System.out.println("=== STRUK PEMBAYARAN KASIR ===");
      System.out.println("Kasir Bertugas : " + this.namaKasir);
      System.out.println("Pesanan        : Meja " + pesanan.getNomorMeja());
      System.out.println("Jumlah Item    : " + jumlah);
      System.out.println("Total Subtotal : Rp " + totalSubtotal);
      System.out.println("Total Pajak    : Rp " + totalPajak);
      System.out.println("TOTAL BAYAR    : Rp " + totalBayar);
      System.out.println("==============================");
   }
}
```

```
package com.mycompany.praktikummgg05.Unguided;

public class Main {
   public static void main(String[] args) {
      Kasir kasir = new Kasir("Budi");

      ItemPesanan item1 = new ItemPesanan("Iced Americano", 18000);
```

```
        ItemPesanan item2 = new ItemPesanan("Beef Croissant", 25000);
        ItemPesanan item3 = new ItemPesanan("Matcha Latte", 22000);

        Pesanan pesanan = new Pesanan(4, 5);

        pesanan.tambahItem(item1);
        pesanan.tambahItem(item2);
        pesanan.tambahItem(item3);

        pesanan.tampilkanPesanan();

        kasir.prosesPembayaran(pesanan);
    }
}
```

Output :

```
=== DAFTAR PESANAN CAFE ===
Nomor Meja: 4
1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)

=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi
Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
TOTAL BAYAR    : Rp 71500.0
==============================
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  8.474 s
Finished at: 2026-10-17T15:53:17+07:00
------------------------------------------------------------------------
```

Penjelasan :

Output dari program Sistem Manajemen Pesanan Cafe ini mencerminkan penerapan konsep relasi antar kelas dalam pemrograman berorientasi objek, yang terdiri dari komposisi, agregasi, dan asosiasi. Pada bagian daftar pesanan, sistem menampilkan nomor meja pelanggan beserta rincian item menu yang dipesan, di mana setiap item menghitung harga dasar dan pajak 10% secara otomatis melalui objek RincianHarga sebagai bentuk hubungan komposisi. Sementara itu, pada bagian struk pembayaran kasir, program menampilkan nama kasir, nomor meja, total subtotal, total pajak, serta total akhir yang harus dibayarkan, yang diproses melalui metode prosesPembayaran sebagai bentuk hubungan asosiasi antara objek kasir dan pesanan.

D. Kesimpulan

- **Konsep Relasi Antar Kelas:** Pemahaman dan penerapan hubungan antar kelas dalam Pemrograman Berorientasi Objek berhasil diimplementasikan, mencakup relasi Asosiasi, Agregasi, dan Komposisi sesuai dengan karakteristik sistem yang dimodelkan.
- **Konsep Relasi Antar Kelas:** Pemahaman dan penerapan hubungan antar kelas dalam Pemrograman Berorientasi Objek berhasil diimplementasikan, mencakup relasi Asosiasi, Agregasi, dan Komposisi sesuai dengan karakteristik sistem yang dimodelkan..
- **Penerapan Agregasi:** Hubungan agregasi diwujudkan melalui kelas Pesanan yang menampung kumpulan objek ItemPesanan secara independen menggunakan array standar, di mana objek komponen dapat berdiri sendiri di luar objek wadahnya.
- **Penerapan Asosiasi:** Hubungan asosiasi diimplementasikan melalui kelas Kasir yang berinteraksi dengan objek Pesanan melalui parameter method prosesPembayaran() untuk menghitung total biaya dan mencetak struk tanpa saling memiliki atribut permanen.
- **Modularitas dan Pengorganisasian Proyek:** Pengelompokan kelas ke dalam *package* terpisah (GuidedAsosiasi, GuidedAgregasi, GuidedKomposisi, dan Unguided) membuat struktur proyek Java menjadi lebih terstruktur, rapi, dan mudah dikelola.